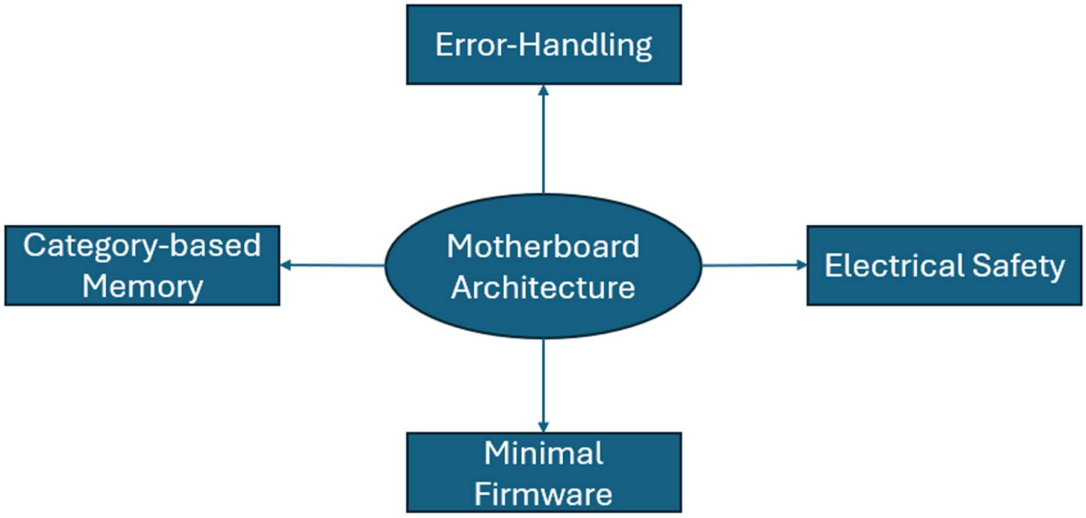
MOTHERBOARD ARCHITECTURE DESIGN

Contents

Introduction

This Motherboard Architecture was designed to improve performance, component safety and efficiency. This architecture was also designed to complete the full hardware stack as the motherboard is the connecting competent between RAM, CPU and more. This architecture was made to address currently limitations and issues with the motherboard. It was also developed as the motherboard is the most important component in any device. This architecture will cover category-based memory, error handling, electric safety, and firmware. Additionally, it will also cover the full workflow of storage to GPU alongside the trade-offs of this architecture.

This Motherboard Architecture cannot promise perfect performance, efficiency or component safety. However, this architecture can promise to improve performance and efficiency. It can also provide electrical safety for components such as the CPU which has specific power requirements. This architecture is just a design that engineers and developers can use as a reference or guide for implementation. Make sure to adapt features to resources and budget.



This spider diagram above shows the 4 main concepts of this Motherboard architecture. We can see category-based memory, error-handling, minimal firmware (BIOS/UEFI) and electrical safety. It does not show all details or concepts involved in this architecture. More details can be found in the section below.

Motherboard Category-Based Memory & Error-handling

- **Category-Based Memory on the motherboard is where RAM modules are split into clear categories.**
- **This means that the RAM slots would be split into OS, App and Background just like they are logically.**
- **This means that there would be dedicated slots for OS-bound RAM, Application RAM and Background Process RAM.**
- **The CPU must also be configured to navigate these RAM modules physically so even if the operating system fails to organise categories it can work effectively.**
- **This improves performance as the CPU knows exactly where to look meaning faster access times meaning less searching.**
- **This also improves compatibility as not only is RAM separation enforced logically but also directly within the hardware.**
- **For error-handling this motherboard architecture requires two changes.**
- **The first change is a hardware acknowledgment protocol.**
- **This is where each component within the device confirms the receipt of data have arrived with all needed details.**
- **This is done by the motherboard chip as it ensures that it receives the signal from all components to ensure that no data has been lost.**
- **If the data were to be corrupted or lost, it is resent by the chip making another request.**
- **For example, the chip may ask for the CPU to fetch the same data from RAM if it is lost.**
- **This improves performance as it allows for less data to be lost and ensures that it is resent if it lost at any point of time.**
- **The second change is PSU error-handling.**
- **This is where the PSU cuts off the voltage automatically without user alert or interaction if the voltage becomes too high for it to handle.**
- **This can be done by the PSU and VRM working together to make sure the voltage doesn't completely fry components such as the CPU within the device.**
- **For example, the CPU has very specific power requirements which if the PSU and VRM cannot provide then it may become damaged or die out completely.**
- **However, this automatic cut off can mean the user loses work immediately as RAM is volatile meaning when power is lost so are the RAM contents.**
- **This is an acceptable trade-off as it protects the device's components allowing the user to still have a working device instead of a non-functional one.**
- **This helps improve electrical safety in 2 ways.**
- **The first way is that it protects components such as the CPU from being damaged or killed by high voltages that they aren't designed to handle.**
- **The second way is that it protects the user as it prevents the device from heating too much to the point it can burn or shock the user.**

Motherboard electrical safety and Minimal Firmware

- **There is one major change that this motherboard architecture requires for electric safety.**
- **This one major change is a hardware-enforced voltage limit.**
- **This is where the VRM limits the CPU to one universal voltage limit that cannot be bypassed by software or BIOS.**
- **This can be done by having the VRM able to provide the CPU with any power requirement if the voltage doesn't exceed the 1.5V limit.**
- **For example, the CPU requests 1.3 or 1.4V which the VRM can provide however if the CPU asks for 5.1V then the VRM does not allow it.**
- **This also means that no throttling is needed because voltage is controlled meaning the CPU doesn't need to slow down as much.**
- **However, users may feel that this is unfair as they may want more control over their device.**
- **This improves electrical safety as it reduces the chances of CPU damage and prevents it from being overloaded by voltage.**
- **This motherboard architecture also supports the usage of minimal firmware such as BIOS/UEFI.**
- **BIOS/UEFI are the boot level instructions needed for initializing hardware and preparing processes for OS loading.**
- **BIOS/UEFI should have a simple GUI interface to allow users to navigate these settings as needed.**
- **These BIOS/UEFI should only focus on the essentials needed for boot and shouldn't be bloated with multiple other features.**
- **Drivers, fan control, network controls, NIC, audio, storage controller and USB can all be managed at the OS or hardware-levels.**
- **BIOS/UEFI should handle security modules initialization while hardware and the OS handle over security features.**
- **For boot support, firmware can have compatibility with secure boot, UEFI boot, legacy BIOS boot and core boot.**
- **Vendors should be only adding simple logos to show the device is booting but there shouldn't be any custom menus or features.**
- **For updates, BIOS/UEFI can be updated securely via the OS who may run an AV scan or use cryptographically signed updates to ensure security.**
- **For code optimization, BIOS/UEFI can be coded using functions for reusable code and to prevent redundant reused lines of code.**
- **However, this minimal BIOS/ UEFI may be bad for businesses are companies cannot promote their brand as much.**
- **This trade-off can be managed by offering other services like firmware updates, system health scans and other functionality for performance.**
- **This improves performance as BIOS/UEFI boot time will be extremely fast and won't be as bloated.**

Full Workflow from Storage to Display

- **First, the user boots up their device for the first time ever.**
- **Second, BIOS/UEFI initialise the hardware, perform the security check and handle over control to the OS in seconds.**
- **Third, The DMA and CPU get the OS instructions from the unformatted internal drive to load up the OS.**
- **Fourth, the OS is loaded into RAM and split into the categories of OS, applications and background processes.**
- **Fifth, the OS sorts out the internal drive into partitions of OS, application and background for faster access.**
- **Sixth, the user configures FDE encryption for the device via TPM meaning that all partitions are encrypted and hardware backed.**
- **Seventh, Cache receives the most frequent OS data in L1 cache, most frequent application data in L2 cache and most frequent background data in L3 cache.**
- **Eighth, VRAM is sorted into clear categories of physics model, textures and frame buffers for quicker access by the GPU.**
- **Ninth, Cores and L1 caches are dedicated to certain categories based on current system workload in real time.**
- **Tenth, the user loads up GTA 6.**
- **The DMA and CPU load GTA 6 instructions and data into the application category of RAM.**
- **The program counter shows App: 0x3A00 as the location of the next instruction inn memory.**
- **The memory address register temporarily stores this location until the address bus uses it.**
- **The address bus navigates to the application RAM slot/category and then finds the address 0x3A00.**
- **The data bus then carries the instruction 0001 00001111 to the MDR which temporarily stores it until it is transferred to the CIR.**
- **The CPU then decodes the instruction 0001 one is decoded into ADD and 00001111 is decoded into R15.**
- **The CPU then executes this instruction using components like the GPU, registers, ALU and external devices for display if needed.**
- **The CPU then writes the result directly back to memory for later use if needed.**
- **The program counter is incremented by one and shows App:0x4B20 as the next location in memory.**
- **Eleventh, the motherboard's chip has been checking for any errors for data corruption or signal loss.**
- **The motherboard has resent data to the CPU a few times due to signal degradation during the fetch process.**
- **The motherboard's VRM and PSU keep voltage for the CPU at the 1.5 limit and keep components of the device safe from high voltage.**

Motherboard Architecture Trade-Offs

- **This motherboard architecture has trade-offs just like any other system I have made.**
- **There also many be more implementation challenges for engineers especially with the physical changes this motherboard architecture can require.**
- **I have identified 5 main trade-offs in this architecture.**
- **The first trade-off is cost.**
- **The trade-off exists as it can be expensive to introduce automatic electrical cut off, hardware-enforced limitations and separate RAM modules.**
- **This trade-off can be managed by focusing on the logical changes first, budget management, testing & validation for hardware and logical changes.**
- **The second trade-off is electrical risk**
- **This trade-off exists as sometimes the voltage can get too high for components within the device.**
- **This means that components like the CPU can be damaged or even die out completely if the voltage is too high.**
- **This trade-off can be managed by the PSU, Automatic power cut for uncontrollable voltage and 1.5V limit enforced by the VRM.**
- **The third trade-off is compatibility.**
- **This trade-off exists as UEFI/BIOS will not be able to support every single boot, and separate RAM modules require new motherboards**
- **This also means that motherboard sockets will not support every CPU or every GPU.**
- **This trade-off can be managed through new generation of motherboards, using existing compatible sockets and supporting the most common boots for the firmware.**
- **The fourth trade-off is UX.**
- **This trade-off exists because the UEFI/BIOS will use a simple GUI interface with no animations or custom menus.**
- **It also exists as users will not be able to control voltage limits as it is a major safety feature for components.**
- **This trade-off can be managed by clearly alerting users, having the GUI interface support control over core security settings and boot configuration and having the operating system manage the main UX features.**
- **The fifth trade-off is business**
- **This trade-off exists as firmware cannot be used for promoting brands anymore which is a key strategy used by companies such as Intel and Lenovo.**
- **This trade-off can be managed by offering performance features built in with the operating system and using this minimal firmware as a marketing strategy.**